

Ailex — AI が書くための中間言語：設計・実験・実装の記録

技術報告 / 2026-07-10

概要

Ailex は、人間ではなく大規模言語モデルが書き手になることを前提に設計したプログラミング言語である。出発点は「AI が開発しやすい言語を作れないか」という問いだった。作業を進める中で、当初の設計アイデアの大半が既存研究にすでに取られていることが分かり、途中で目標を「新規性のある研究」から「実際に使える言語を作ること」へ移した。本書は、その設計判断・実験結果・実装状況を、否定的な結果も含めてそのまま残す。

1. 設計の出発点

人間向けの言語は読み書きのしやすさを最適化してきた。書き手が LLM になると、ボトルネックは可読性ではなく「もっともらしく間違えること」に移る。そこで Ailex の設計目標を、生成と検証の力学に置いた。狙いは三つある。間違ったプログラムが出力しにくいこと。間違いをすぐ機械が指摘して直せること。理解に必要な情報が近くにまとまっていること。

構造は三層で考えた。L0 が意味グラフ（正）、L1 が一意な正規形（モデルの生成面）、L2 が人間向けの表示。当初の中核アイデアは四つ。左から右へ生成するとの途中状態も型が付く「単調型付け可能性」、検証器の返答を機械可読データとして生成に戻すこと、内容アドレス化した三層分離、型で穴を段階的に埋める合成。

2. 先行研究の監査

四つのアイデアを既存研究に当てた結果は厳しかった。

アイデア	最も近い先行研究
型安全な制約付き生成	ETH "Type-Constrained Code Generation" (PLDI 2025, arXiv:2504.09246)
型付き穴+型文脈を LLM に渡す	Hazel / ChatLSP (OOPSLA 2024, arXiv:2409.00921)
構文レベルの制約デコード	SynCode (arXiv:2403.01632)
内容アドレス化	Unison ほか
意図の形式化（指示層）	Intent Formalization grand challenge (arXiv:2603.17150)、TiCoder
層をまたぐ早期検証	VERIMAP ほか

単一の「世界初の機構」は、この分野が 2024-2026 に急速に埋まったため、現実的に取れない。指示の形式化は grand challenge として名前が付くほど活発で、層をまたぐ早期検証も VERIMAP がほぼ同じことをやっている。

3. 実験

設計の可否を、Anthropic API を使って実データで測った。

実行層の比較（Ailex のマスク生成 vs Python の生成＋事後修復）

10 個の課題を、同じモデルで両方式に解かせ、隠しテストで採点した。Opus

では両方式とも満点で差が出ない（天井効果）。Haiku では Ailex 側 27/30、Python 側 30/30。Ailex 側は入力トークンが 7～9 倍かかった。読み取れたのは、型による制約は構文と型の誤りを消すが、意味の誤りは消さないこと。そして、モデルが強い、または課題が既知だと、言語側の介入はほとんど効かないこと。

フィードバックの実験

検証器の返答を「自然言語」で返すか「構造化データ」で返すかで、修復の効率が変わるかを測った。スコープ（使える名前と型）を伏せた条件で、Haiku に 50 回ずつ解かせた。

条件	隠しテスト合格
自然言語（誤りのみ）	40 / 50
自然言語（スコープを散文で列挙）	50 / 50
構造化 JSON（同じ内容）	50 / 50

差は完全に「スコープ情報を足したかどうか」で決まり、「JSON か散文か」は寄与ゼロだった。効いたのは、モデルが未知の関数名（dot）を当てられず発散するとき、「何が使えるか」を教えると一手で回復する、という機構。これは当初の中核アイデア「機械可読な構造化フィードバック」を否定する結果でもある。価値は形式でなく情報内容にあり、その情報効果は Hazel の先行結果と重なる。

4. 方針転換

四つの中核が個別に取られ、フィードバックの一つは実験で否定された。ここで、目標を「新規性のある研究」から「実際に使える言語」へ移した。Go も Python も、新しい機構で有名になったのではなく、よく設計されて使いやすかったから広まった。基盤としての価値は、部品の新しさでなく、まとまりと動くことにある。成功の定義を「世界初」から「使える・よく設計されている」に置き換えた。

5. 実装

実在の言語がどう作られたかを調べ、その手順に沿った。Go は構想が 2007 年、1.0 が 2012 年、自己ホストは 2015 年の 1.5。最初のコンパイラは別言語（C）で書いた。定番の順序は、まず木を歩くインタプリタを作り、早い段階で大量のテストを用意し、二つ目のバックエンドは同じテストで振る舞いの一致を担保する。

この順序で v0.1 コアを新規に実装した。字句解析・再帰下降パーサ・双方向型検査・評価器・構造化診断・契約の実行・JavaScript への変換・コマンドライン。実測を設計に反映した判断が二つある。中置演算子を採用したのは、実験でモデルが $a * b$ を自然に書き、関数呼び出しの強制が構文エラーを連発したため。型エラーの診断がスコープを名前と型で開示するのは、フィードバックの実験で一番効いた情報を言語機能に昇格させたもの。

実際に書いて痛点を洗った（ドッグフーディング）。統計・範囲制限・成績判定は素直に書けた。再帰と前方参照は、仕様では想定していなかったが実装で自然に動いた（仕様を訂正）。一方、高階関

数は型検査を通るのに一方のバックエンドで落ちる不健全なバグが見つかった。まず型検査で関数値を禁止して両バックエンドを揃え、次の版で正式にサポートした。

v0.2 で無名関数・第一級関数・`map / filter / fold`を入れた。これでリストの変換が書けるようになった。ドッグフーディングで書けなかったベクトルのスカラ倍や要素の絞り込みが、いま動く。

6. 現状

適合テストは 36 件すべて緑。インタプリタと JavaScript の二つのバックエンドが、同じテストで一致する。書いたプログラムは本物の JavaScript に変換されて動く。数値・真偽・文字列・リストの型、中置演算子、局所束縛、契約、再帰、高階関数、リスト処理まで書ける小さな言語になった。

限界は隠していない。課題台がまだ浅い。構造データ（レコード）・副作用・モジュールは未対応。採用と、AI がこの言語を学習していない（学習データがゼロの）問題は別の山として残る。規模としてはまだ個人プロジェクトである。

7. 次の候補

構造データ（レコード）を入れると、名前付きのまとまりを扱う普通のプログラムが書けるようになる。あるいは、v0.2 を実際に書いて次の痛点を証拠で洗ってから決める。

付録：主なファイルと実行

- ・言語仕様: `ailex/SPEC.md` / 設計判断: `ailex/LANGUAGE.md` / 手順: `ailex/PROCESS.md`
- ・実装: `ailex/core/lang.ts` (コア), `tojs.ts` (JS 変換), `cli.ts` (CLI), `conformance.ts` (テスト)
- ・実験記録: `ailex/EXPERIMENTS.md` / 先行研究監査: `ailex/PRIOR_ART.md`, `NOVELTY.md` / ドッグフーディング: `ailex/DOGFODD.md`
- ・実行例: `node ailex/core/conformance.ts / ./ailex/ailex run ailex/examples/lists.ax`